

Analytics Database

The first analytics database implementation uses SQLite with SQLAlchemy ORM models. The database is generated from archived JSON files and can be rebuilt at any time.

Files

- `backend/database.py`: SQLAlchemy engine, session factory, SQLite PRAGMA setup.
- `backend/models.py`: ORM models for seasons, divisions, source files, teams, players, matches, races, results, tracks, and penalties.
- `backend/import_json_to_db.py`: import/rebuild command for archived JSON files.
- `backend/inspect_db.py`: small inspection command for counts and import review rows.
- `backend/data/team_aliases.csv`: import-time team tag cleaning map.
- `backend/data/ctc_stats.sqlite`: generated SQLite database.

`backend/data/*.sqlite` is ignored by git because the database is generated output. The archived JSON files are the source of truth.

Install Dependencies

From `backend/`:

```
python -m pip install -r requirements.txt
```

`SQLAlchemy` is now included in `backend/requirements.txt`.

Rebuild The Database

From `backend/`:

```
python import_json_to_db.py --rebuild
```

By default this reads:

```
backend/JSON/
```

and writes:

```
backend/data/ctc_stats.sqlite
```

The importer expects archived match files to live under:

```
backend/JSON/{league}/{season}/{division}/
```

Example:

```
backend/JSON/ctc/s1/d4/W1 F_s u.json
```

Inspect The Database

From `backend/`:

```
python inspect_db.py
```

Current initial import summary:

```
seasons: 1
divisions: 3
source_files: 96
teams: 41
team_season_entries: 41
players: 180
matches: 96
match_teams: 193
match_players: 972
tracks: 151
races: 1138
race_player_results: 11519
penalties: 34
```

The importer marked one match as needing review:

```
W8 6c 0 - Expected 2 teams, found 3 raw team objects.
```

That matches the project rule that CTC matches should resolve to two real teams. A third raw team is likely a sub/team parsing anomaly.

Import Behavior

The importer:

- uses SQLAlchemy ORM models
- creates SQLite with WAL mode enabled
- prefers `.json` files over same-stem `.txt` files

- stores source file hashes for deduplication
- infers league, season, and division from folder path
- applies team aliases from `backend/data/team_aliases.csv`
- parses week number from filenames like `W1 ...`
- stores full raw match JSON on the `matches` row
- keeps team/player/track aliases and raw names
- expands race scores and positions into `race_player_results`
- infers `bagger` when race score is `1`
- marks non-two-team matches as `needs_review`

Team Tag Cleaning

Some raw Table Bot files misparse complicated clan tags into partial tags such as one-letter fragments, `No Tag`, or a trailing fish-symbol character. These are corrected at import time by `backend/data/team_aliases.csv`.

The alias file is match-aware. This matters because a raw tag like `M` can mean different teams in different matches. The importer first checks for an exact row:

```
league_code, season_code, division_code, match_label, raw_team_key
```

Then it falls back to a division-wide row where `match_label` is blank.

The raw JSON and raw table fields are still preserved for auditability. For example, a player row may still show `tag_raw = No Tag`, but its `match_team_id` and `team_season_entry_id` point to the corrected canonical team.

After the first cleaning pass, the rebuilt database has:

```
teams: 39
team_season_entries: 54
matches needing review: 0
```

Next Steps

- Add an upload/admin endpoint that saves raw files to the archive path.
- Add import locking so only one upload import runs at a time.
- Add review tooling for `needs_review` matches.
- Add analytics API endpoints that read from SQLite instead of CSV.
- Add migrations with Alembic once the schema settles.